ЗАДАНИЕ №5
Этап 5. Шумоподавление и извлечение признаков — автоэнкодеры (Autoencoders)

Цель

Разработать и обучить автоэнкодер для очистки зашумленных сигналов (имитация сбоя сенсоров) и оценить, как предварительная обработка автоэнкодером влияет на качество классификации из этапа 4.

ЗАДАЧИ

1. **Добавление искусственного шума**
   - Взять тестовую выборку (или часть данных) и добавить к каждому признаку гауссов шум с нулевым средним и небольшой дисперсией (например, `np.random.normal(0, 0.1, size=...)`).
   - Убедиться, что зашумленные данные визуально отличаются от исходных.

2. **Построение и обучение автоэнкодера**
   - Создать модель автоэнкодера. Можно использовать полносвязную архитектуру или сверточную (1D ConvAutoencoder), если работаем с окнами.
   - Для полносвязного: входной слой (размер = длина окна × количество признаков), скрытые слои уменьшающейся размерности (например, 256 → 128 → 64) — это кодировщик, затем симметричные слои для декодирования.
   - Функция потерь: `mse`.
   - Обучить автоэнкодер на чистых данных (вход и выход — одни и те же данные). Использовать валидацию.

3. **Очистка зашумленных данных**
   - Подать зашумленные данные на вход обученному автоэнкодеру, получить восстановленные (denoised) данные.

4. **Оценка влияния на классификацию**
   - Взять модель-классификатор из этапа 4. Применить её к трём вариантам тестовых данных: исходным чистым, зашумленным, восстановленным автоэнкодером.
   - Сравнить метрики качества классификации (accuracy, F1) для этих трёх вариантов. Построить диаграммы.
   - Сделать вывод о способности автоэнкодера повышать робастность модели.

5. **Визуализация**
   - Показать несколько примеров исходного сигнала, зашумленного и восстановленного (например, график одного признака по времени).

6. **Сохранение модели**
   - Сохранить обученный автоэнкодер (`autoencoder.h5`).

ВХОДНЫЕ ДАННЫЕ

- Обработанные данные, модель классификатора из этапа 4.

Результат этапа

- Jupyter Notebook с кодом автоэнкодера и анализом.
- Сохранённая модель автоэнкодера.

ОБЩИЕ ТРЕБОВАНИЯ К ВЫПОЛНЕНИЮ ПРОЕКТА

- Все этапы должны быть реализованы на языке Python с использованием библиотек Keras/TensorFlow, pandas, numpy, matplotlib, scikit-learn.
- Для каждого этапа рекомендуется создавать отдельную папку или notebook.
- На каждом этапе необходимо сохранять промежуточные результаты (модели, обработанные данные) для использования в следующих этапах.
- Отчёты по этапам сдаются в виде Jupyter notebook с подробными комментариями и выводами. Notebook должен быть исполняемым (можно проверить результаты).

КРИТЕРИИ ОЦЕНКИ

- Корректное построение и обучение автоэнкодера.
- Эксперимент с шумом и оценка влияния на классификацию.
- Визуализация восстановления сигнала.
- Обоснованные выводы.